Find My Mines

Find the bombs. Read the clues. Beat the clock.

2 players · 36 cells · 11 bombs · 10 seconds

A real-time multiplayer game with AI practice and replay coaching

The assignment

Two players. Two computers. One networked game.

Assessment area	Points	Presentation evidence
Preparation & creativity	5	Live demonstration
Fundamentals	10	Game, network, controls
Extra features	≤ 10	AI and supporting features

AI is required. Extras count after the fundamentals are complete.

The game in one minute

1. Two players join a room; the server chooses who starts.
2. A covered **6 × 6 board** contains **11 randomly placed bombs**.
3. Reveal a bomb: **+1 point**, and keep the same turn deadline.
4. Reveal an empty cell: see its adjacent-bomb count; play passes.
5. Time runs out after **10 seconds**: play passes automatically.

Find all 11 bombs → higher score wins → both players can rematch.

From welcome to rematch

Choose a nickname or sign in

↓

Create a room • Join a room • Find opponent

↓

Welcome message • Player names • Scores • Live countdown

↓

Win / Lost • Final scores • Two-player rematch vote

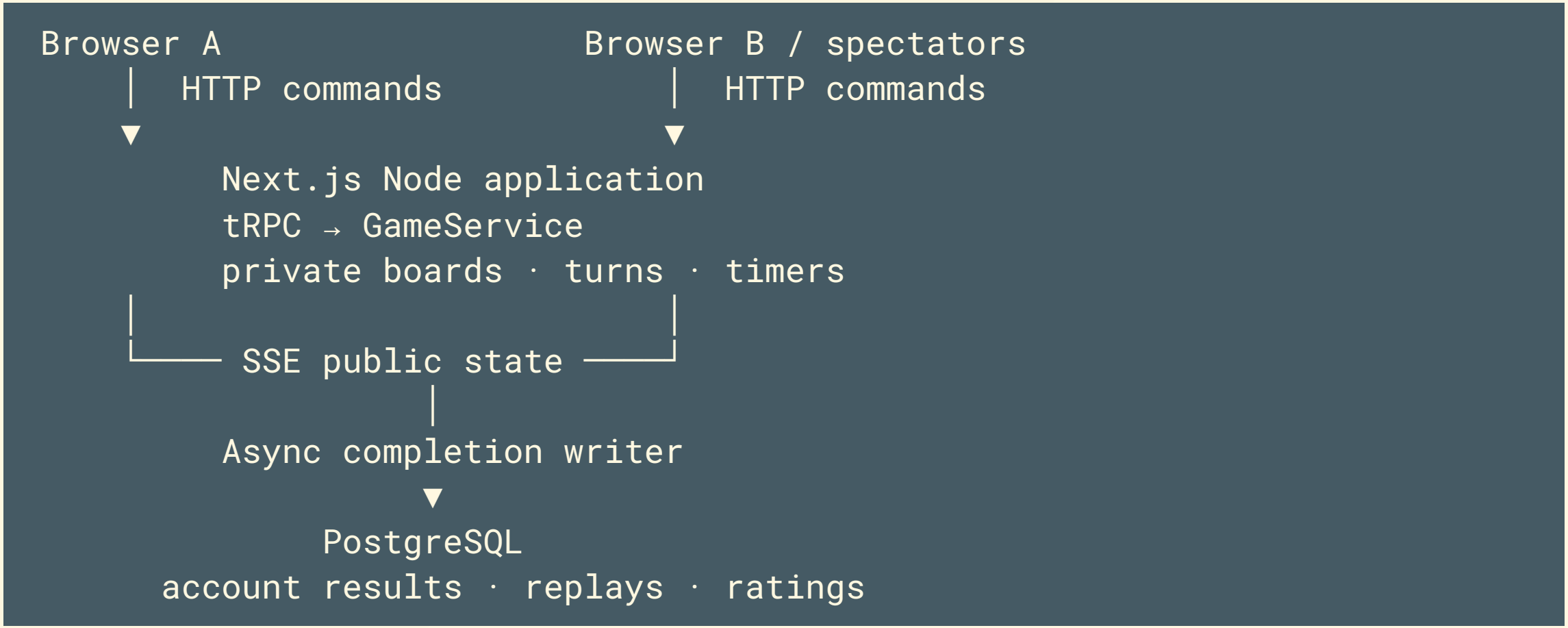
The previous winner starts the rematch on a fresh board.

One server owns the truth

Browsers request moves.
The server validates them, advances the game,
and publishes the same public state to every viewer.

The hidden board never leaves the server.

Architecture



Networking: the implementation choice

Direction	Transport	Purpose
Client → server	HTTP / tRPC	Join, reveal, rematch, reset
Server → client	SSE / tRPC	Lobby, room, presence

Reconnects receive full snapshots; revisions reject stale updates.

Brief alignment: Uses HTTP + SSE, without raw sockets or WebSockets. Instructor acceptance is required.

Fair turns, even with delayed clients

- **Server deadlines** advance play without browser requests.
- **Match and turn IDs** reject actions from an earlier game or turn.
- **Synchronous transitions** validate and apply a move together.
- **Signed, HTTP-only cookies** establish identity and seat ownership.
- **Presence leases + reconnect grace** handle interrupted connections.

Finding a bomb keeps the original deadline; it does not add time.

Operator controls

The server dashboard makes the match observable.

- See connected game-session count and client nicknames.
- Inspect rooms, queue size, scores and pending saves.
- Reset one room: clear its active game and scores.
- Start again when two players are connected.

Access requires a verified **administrator account**; commands and subscriptions enforce authorization on the server.

Two ways AI supports play

Practice against a bot.

Learn from a completed match.

Both use visible game information, without hidden bomb positions.

AI feature 1 · Practice opponent

Visible board + clues + legal moves → model → validated move

- Uses the same board, scoring rules and 10-second turns.
- Receives no hidden bombs, player identities or message-board text.
- Invalid or failed early responses use a random legal fallback.
- Deadline expiry cancels pending work and passes the turn.

Practice rooms are unlisted, unrated, and have no saved replay.

AI feature 2 · Replay coaching

Saved public replay → anonymized evidence → saved analysis

- A match participant requests analysis after an account game.
- The report combines calculated statistics with AI coaching.
- Event links jump to the relevant moment in the replay.
- Both players' consent is required for public viewing.

Coaching is an interpretation of visible play, not proof of optimal moves.

Beyond the required match

Extra feature	Value to players
Public and invite-only rooms	Choose how to meet an opponent
Matchmaking	Find another waiting player
Spectating	Watch a live game without taking a seat
Friends and invitations	Challenge an online friend
Global message board	Talk across the lobby and rooms

Progress that lasts beyond one game

Feature	Current behavior
Match history	Saved results for two-account matches
Replay playback	Seek, step, play/pause, and 1× / 2× / 4× speed
Achievements	15 badges with separate sharing consent
Elo leaderboard	Start at 1000; rated account matches

Public history and replays require **both participants** to opt in. Guest and mixed matches do not create saved game records.

Stack and deployment boundaries

Next.js App Router · React · TypeScript
tRPC + SuperJSON · Better Auth · Drizzle + PostgreSQL
Tailwind CSS 4 · DaisyUI 5

- One persistent Node process serves all rooms.
- Live rooms, queues and unsaved results live in memory.
- Saved account data survives in PostgreSQL.
- Restarting clears live state; multiple replicas are unsupported.

Demonstration · Two computers

Computer A	Computer B
Runs the Node server	Runs a browser client
Opens a player browser	Opens the same configured origin

1. Enter nicknames and show both clients in the dashboard.
2. Reveal cells; compare scores, clues and turn changes.
3. Let a turn expire, finish the game, then agree to rematch.
4. Reset from the dashboard and show both scores cleared.

Demonstration · Extra features

1. **Bot practice:** show a legal AI move within a normal turn.
2. **Account match:** finish a game and open its saved replay.
3. **AI analysis:** open coaching and jump to a cited event.
4. **Social play:** send a friend invitation and enter the room.
5. **Progress:** show the leaderboard and earned badges.

Prepare two accounts, provider access, and available bot quota.
Keep a saved replay ready for the coaching demonstration.

Verification and requirement alignment

Area	Evidence / remaining check
Rules, timing, authorization	Behavioral tests
Lint and TypeScript	`pnpm check`
Live HTTP + SSE	Empty-lobby test
Two-computer operation	Live demonstration
Socket / address wording	Instructor acceptance

Live deployment and grading acceptance remain separate checks.

Ready to find the last bomb?

Timed competition. Shared live state. AI-assisted learning.

Live demo & questions